

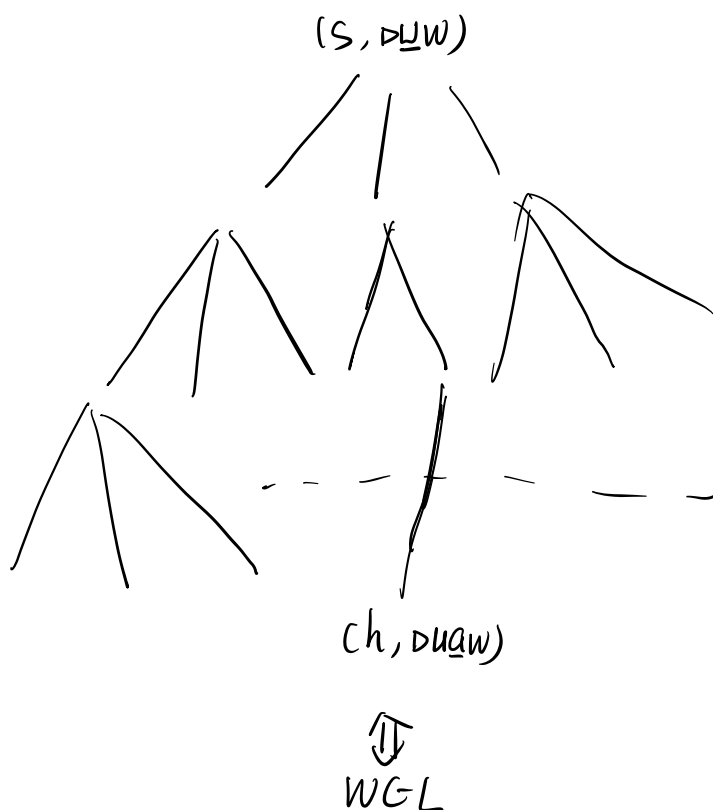
Theorem

Any NTM can be simulated by a DTM.

Proof:

NTM semidecides $L \Rightarrow$ DTM semidecides L .

input w

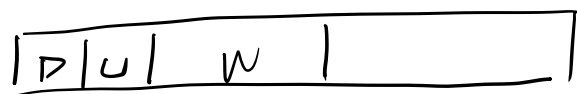


BFS

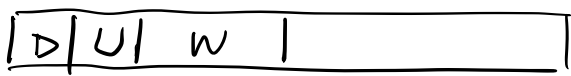
$$\Delta \subseteq (K - H) \times \Sigma \longrightarrow K \times (\Sigma \cup \{\leftarrow, \rightarrow\})$$

$$r = |K| \times (|\Sigma| + 2)$$

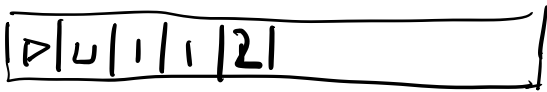
3-tape DTM



store the input.

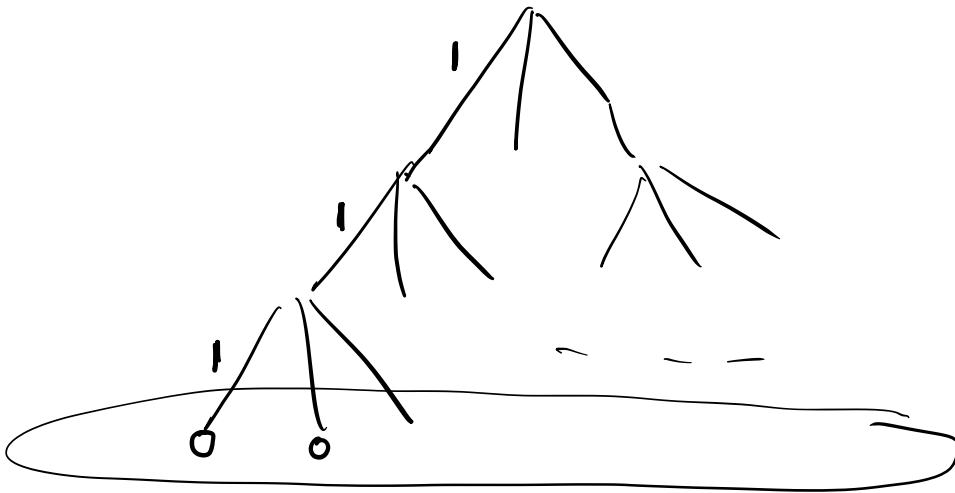


simulate NTM on w



enumerate strings over $\{1, 2, \dots, r\}$

1 1 1 1



Church - Turing Thesis

Intuitive notion
of algorithms

equals

Turing machines that
halt on every input.

solve.
decision
problems

decide.
languages

1. formal description $M = (K, \Sigma, \delta, S, H)$

2. Implement-level description diagram

3. high-level description pseudo code

algorithm
↑
input: object

TM
↑
input: string

Fact.

Every object O can be encoded into ^a string, denoted " O ".

Encoding of $O_1, \dots, O_n$ is denoted " O_1 " " O_2 " " O_3 " ... " O_n ".

Example

Even a TM can be encoded into a string.

$M = (K, \Sigma, \delta, s, H)$

↓

a string over $\{a, q, 0, 1, ', (,)\}$

• encoding states

$$i = \min_j \{j \mid 2^j \geq |K|\}$$

$$q \in \{0, 1\}^i$$

$$s = q 0^i$$

• encoding alphabet $\{ \leftarrow, \rightarrow \}$

$$t = \min_j \{j \mid 2^j \geq |\Sigma| + 2\}$$

$$a \in \{0, 1\}^t$$

$$\sqcup : a \sigma^{\dagger}$$

$$\triangleright : a \sigma^{\dagger 1}$$

$$\leftarrow : a \sigma^{\dagger 2} 10$$

$$\rightarrow : a \sigma^{\dagger 2} 11$$

$$\delta : (K-H) \times \Sigma \rightarrow K \times (\Sigma \cup \{\leftarrow, \rightarrow\})$$

$$(q_{00}, a_{100}, q_{01}, q_{00}), (q, \dots, \dots, \dots), \dots$$

H:

↑
"M"

Fact:

Given an valid encoding "O", a Turing machine is always able to decode it.

Example

Problem

Given a graph G , is G connected?

↪ is "G" in L

Language

$L = \{ "G" : G \text{ is a connected graph?} \}$ ← decide.

$M =$ on input "G"

0.1 If the input is not a valid encoding of graphs,
reject.

by default

0.2 decode "G"

1. select a node of G and mark it
2. repeat the following until no new node are marked
for each marked node,
mark all its neighbors
3. If all nodes of G are marked.
accept "G"
4. else
reject.

Decidable problem \ recursive language

About DFA \ NFA \ regular languages

RI Problem

Given a DFA B and a string w , does B accept w ?

Language

$A_{DFA} = \{ \langle B, w \rangle : B \text{ is a DFA that accepts } w \}$

$\updownarrow$
 $\langle B, w \rangle \in L?$

$M_R =$ on input $\langle B, w \rangle$

default

1. simulate B on w
2. If B accepts w ,
accept " B " " w "
3. else —
reject " B " " w "

$R1^*$ Problem (for a fixed DFA B)

Given a string w , does B accept w ? $\Leftrightarrow$ is $w \in L(B)$?

Language
 $L(B)$

$\rightarrow$ regular $\subseteq$ recursive.

M_{R1^*} = on input w

1. run M_{R1} on " B " " w "
2. If M_{R1} accepts " B " " w "
accept w
3. else
reject.

$R2$ Problem

Given a NFA B and a string w , does B accept w ?

Language

$A_{NFA} = \{ "B" "w" : B \text{ is a NFA that accepts } w \}$

M_{R2} = on input " B " " w "

1. convert B into an equivalent DFA D .
2. run M_{R1} on " D " " w "
3. If M_{R1} accepts " D " " w "
accept " B " " w "
4. else
reject.

Reduction: NFA $\longrightarrow$ DFA

$R3$ language

$$A_{PEX} = \{ "R" "w" : R \text{ is regular expression and } w \in L(R) \}$$

M_{R3} = on input " R " " w ".

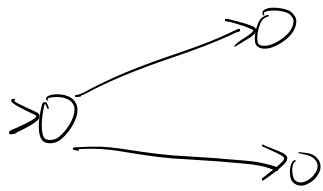
1. convert R into ^{an} equivalent NFA B
2. run M_{R2} on " B " " w "
3. If M_{R2} accepts " B " " w ",
accept " R " " w "
4. else
reject.

$R4$ Problem (does B accept any string?)

Given a DFA B , is $L(B) = \emptyset$?

language.

$$\bar{E}_{DFA} = \{ "B" : B \text{ is a DFA and } L(B) = \emptyset \}$$



0

0

0

M_{P4} = on input "B"

"

"

RS Problem

"

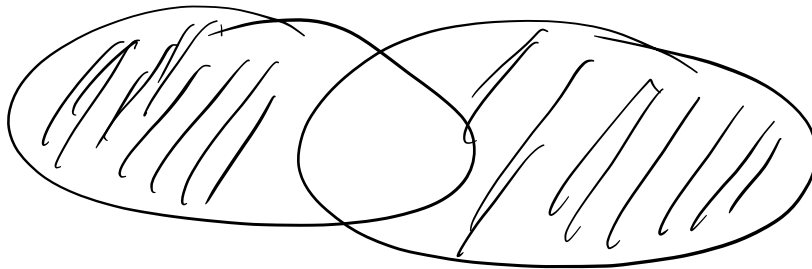
"

Language.

$EQ_{DFA} = \{ \langle A, B \rangle : A \text{ and } B \text{ are two DFAs, } L(A) = L(B) \}$

Symmetric difference.

$$L(A) \oplus L(B) = \{ w : w \in L(A) \cup L(B) \text{ and } w \notin L(A) \cap L(B) \}$$



Observation

DFA C

$$1. L(A) \oplus L(B) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

$$2. L(A) = L(B) \text{ if and only if } L(A) \oplus L(B) = \emptyset \Leftrightarrow L(C) = \emptyset$$

M_R = on input "A" "B"

1. construct a DFA C st $L(C) = L(A) \oplus L(B)$
2. run M_R on " C "
3. If M_R accepts " C ",
accept "A" "B"
4. else
reject.

About CFL

Definition

A CFG $G = (V, \Sigma, S, R)$ is in Chomsky normal form (CNF) if every of its rule is in the following forms.

1. $A \rightarrow BC$ $A, B, C \in (V - \Sigma)$
2. $A \rightarrow a$ $A \in (V - \Sigma), a \in \Sigma$
3. $S \rightarrow \epsilon$
4. S does not appear on the RHS of a rule. ✓

Theorem equivalent
Every CFG has an $\vee$ CFG in CNF.

Proof

Given $G = (V, \Sigma, S, R)$, convert it to CNF.

① add a new start symbol S_0
a new rule $S_0 \rightarrow S$.

② fix ϵ -rules

$A \rightarrow \epsilon$ with $A \neq S_0$

③ fix short rules

$$A \rightarrow B \text{ with } A, B \in (V - \Sigma)$$

④ fix long rules

$$A \rightarrow u \text{ with } u \in V^*, \text{ and } |u| > 2$$

⑤ fix $A \rightarrow u_1 u_2$ with u_1 or u_2 being terminals

$$\begin{array}{ccc} A \rightarrow Ba & & A \rightarrow A^* B^* \\ \textcircled{A \rightarrow ab} & \swarrow \searrow & A^* \rightarrow a \\ A \rightarrow aB & & B^* \rightarrow b \end{array}$$

fix $A \rightarrow e$

$$\textcircled{B \rightarrow aAbAc}$$

$$B \rightarrow abAc$$

$$B \rightarrow aAbc$$

$$\text{---} \cancel{B \rightarrow ab}$$

$$B \rightarrow abc$$

fix $A \rightarrow C$

$$B \rightarrow aAbAc$$

$$B \rightarrow aCbCc \quad B \rightarrow aCbAc$$

$$B \rightarrow aAbCc$$

fix $A \rightarrow u_1 u_2 u_3 \dots u_k$ $k \geq 3$

$$A \rightarrow u_1 u_2$$

$$U_2 \rightarrow u_2 U_3$$

$$U_3 \rightarrow u_3 U_4$$

...

...

...

$$U_{k-1} \rightarrow u_{k-1} U_k$$

$$U_k \rightarrow u_k$$

Observation

If CFG in CNF generates a string of length $n \geq 1$, it must take exactly $2n-1$ steps.

C1. $A_{CFG} = \{ \langle G, w \rangle : G \text{ is a CFG that generates } w \}$

M_{C1} = on input $\langle G, w \rangle$

1. convert G to G' in CNF

2. If $|w| = 0$,

If G' has $S \rightarrow \epsilon$

accept $\langle G, w \rangle$

else

reject.

3. If $|w| \geq 1$,

list all the derivations of length $2|w|-1$

↑

finite, $\leq |R|^{2|w|-1}$

4. If any listed derivation generates w ,
 accept " G " " w "
 else
 reject